

Token da RD sem Baserow e conversão RD por último no set-tier

Duas mudanças aplicadas em produção no n8n: o armazém do access_token da RD Station saiu do Baserow e virou Data Table nativa, e a automação de tier foi reordenada para que nenhum lead deixe de chegar ao vendedor por instabilidade da RD.

22/07/2026

Data da aplicação

Growth — RevOps

Área

n8n.cardapioweb.com

Ambiente

CONTEXTO

O problema em duas frentes

1. O token da RD morava numa ferramenta de terceiro

Toda automação que fala com a RD Station precisa de um `access_token` OAuth que expira em ~24h. Esse token ficava guardado numa tabela do **Baserow** (ferramenta externa): o workflow `[UTIL] Refresh token Rd Station` validava/renovava o token e gravava lá, e cada automação lia essa tabela antes de chamar a RD. Isso criava uma dependência externa no caminho crítico de **todas** as integrações com a RD — indisponibilidade do Baserow = automação parada.

2. No set-tier, a RD ficava na frente do vendedor

Na automação que define tier e envia leads para a Meetime (`[SQL] Automação para definir tier...`), a conversão na RD rodava **antes** do envio à Meetime. Qualquer erro da RD (instabilidade ou payload rejeitado) interrompia a execução — e o lead não chegava ao vendedor por causa de uma etapa que é só registro de marketing.

EVIDÊNCIA QUE MOTIVOU A REORDENAÇÃO

Das execuções com erro do set-tier no período de 20 a 22/07 (7 execuções), **7 de 7 falharam no trecho token/RD**, todas antes do envio à Meetime: 4 no Execute Workflow do token ("Service unavailable"), 1 na leitura do token, e 2 na conversão RD com "Bad request" — estas últimas são erro de payload, que **nenhum retry resolve**; só tirar a RD do caminho do lead protege a entrega.

Fonte: API pública do n8n — `GET /api/v1/executions?workflowId=GADrDhIEi1NvOJn9&status=error`, últimas 25 execuções com erro, nó de falha extraído de `resultData.lastNodeExecuted`. Janela citada: 20–22/07, horários em Brasília.

Migração do armazém do token

Foi criada a Data Table nativa do n8n `rd_station_token` (id `n4sEAnrMqvG70nHp`) com uma única coluna, `Access_token` — **mesmo nome do campo que existia no Baserow**, de propósito: as expressões `$json.Access_token` dos workflows continuaram funcionando sem nenhuma edição. A troca em cada workflow foi só do nó (Baserow → Data Table), preservando nome, posição e conexões.

766

workflows varridos na instância para mapear quem usava o token

22

workflows ativos migrados (21 consumidores + o UTIL que renova)

30

nós Baserow substituídos por Data Table

Ordem de aplicação (piloto antes do lote)

- **Piloto:** `[PAR] Criar conversão Cw Apps` — aplicado, inspecionado e aprovado antes do restante.
- **Dupla crítica:** `[UTIL] Refresh token Rd Station` (leitura e gravação do token) + set-tier, em sequência imediata para não abrir janela de inconsistência.
- **Lote final:** os 19 demais (Etapas 1–5 reopen, Associação de Etapa, CQL Meetime/RD, TQL, LP de parcerias, indicação/adicação manuais, clientes ganhos, compras futuras SIM/NÃO, reopen frios, Reopen SDR AI) — 19/19 verificados.

Um detalhe de arquitetura descoberto no mapeamento

Existem **dois sistemas independentes de token RD** na instância, com apps RD distintos: o do Baserow (alvo desta migração) e o `[UTIL] Obter credenciais da RD Station`, que grava no Postgres do CDP e atende [IMP], [PAR] ganhos e [REV] reajuste. O segundo **não foi tocado**.

VALIDAÇÕES EXECUTADAS

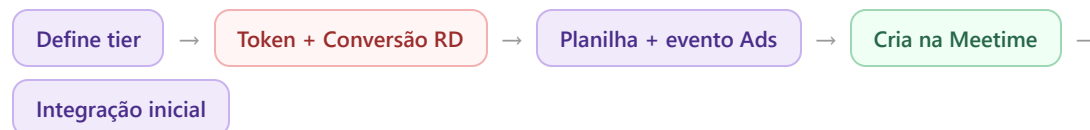
- Formato dos nós Data Table (get, update e delete) validado **empiricamente** em workflows de teste descartáveis, contra linha sandbox — nunca contra a linha de produção. Testes apagados em seguida.
- Prova end-to-end: workflow de teste chamou o UTIL real pós-troca (do mesmo jeito que os consumidores chamam) e recebeu o token válido de 670 caracteres lido da Data Table.
- Varredura final: **zero** workflows ativos lendo a tabela antiga do Baserow; **zero** execuções com erro na instância no pós-troca imediato.
- Backup do JSON original dos 22 workflows guardado antes de cada alteração.

Fonte: API pública do n8n — varredura por `GET /api/v1/workflows` (nós do tipo `baserow` com `tableId 559701`), verificação pós-write por releitura de cada workflow e comparação de nós/conexões contra o backup.

Conversão RD movida para o fim

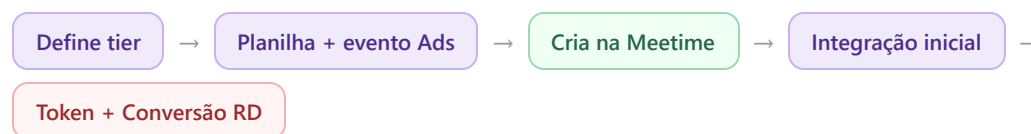
A reordenação foi aplicada nos dois braços do set-tier (leads de formulário e leads de WhatsApp). O trio token + chamada RD foi movido em bloco para **depois** do último passo da cadeia de entrega. Nenhum nó pós-RD dependia da resposta da RD (todas as referências são nomeadas), o que tornou a mudança segura.

ANTES



RD falhou → execução para → lead não chega à Meetime.

DEPOIS



RD falhou → lead já está com o vendedor; o erro fica visível no log para recuperar a conversão depois.

Decisões tomadas junto com a reordenação

- **Retry automático (3 tentativas, 5s de espera)** nos 6 nós de token — os 3 nós de chamada RD já tinham retry configurado. Cobre a instabilidade transitória ("Service unavailable").
- **Erro continua visível:** optou-se por manter a falha da RD como erro de execução (em vez de "engolir" com continue-on-fail), para que conversões perdidas sejam auditáveis e recuperáveis em lote.
- **Sem desvio para o form EUA:** o lead do formulário "LP - GOOGLE - PAGO EUA" só recebia a conversão RD (era barrado da Meetime pelos filtros). Com o novo desenho, esse lead deixa de gerar conversão na RD — decisão consciente de simplificar o fluxo.
- **Braço do botão de WhatsApp** (terceira chamada RD) já era terminal no desenho antigo e não foi movido; ganhou o mesmo retry.

O desenho foi validado primeiro numa **cópia de teste inativa**, revisada e ajustada manualmente antes da réplica no principal. Na réplica, dois pontos foram corrigidos de volta: o nó Webhook de entrada (removido por engano na edição do teste) foi preservado no principal, e o filtro dos nós de token manteve a chave `id = 1` que o editor visual havia descartado ao salvar.

Fonte: workflow `GADrDhIEi1NvOJn9` ao vivo, diff completo teste × principal por nó e por conexão antes da aplicação; verificação pós-write com releitura (109 nós, webhook `set-tier` intacto, novas conexões conferidas uma a uma).

RESULTADO

Como o sistema fica depois das duas mudanças

Dimensão	Antes	Depois
Armazém do token RD	Tabela no Baserow (serviço externo, credencial própria, ponto único de falha fora de casa)	Data Table nativa do n8n <code>rd_station_token</code> — sem dependência externa, mesma infraestrutura dos workflows
Falha da RD no set-tier	Interrompia a execução antes da Meetime — lead não chegava ao vendedor	Lead entregue primeiro; conversão RD é o último elo e falha isolada, visível no log
Instabilidade transitória	Erro na primeira tentativa derrubava o fluxo	Retry automático 3× com 5s nos nós de token e RD
Auditoria	—	Backups JSON de todos os workflows alterados; testes descartáveis documentaram cada formato de nó antes do uso em produção

O que observar nos próximos dias

- **Primeira renovação automática do token** na Data Table (o token vive ~24h; a primeira renovação pós-migração confirma o ciclo completo de escrita).
- **Primeiras execuções reais do set-tier reordenado** — conferir leads chegando à Meetime e conversões RD registradas na sequência.
- Se aparecer erro de conversão RD, o lead **não é impactado** — a execução com erro fica no log com os dados necessários para reenviar a conversão.

INVENTÁRIO DAS ALTERAÇÕES

Data Table: `rd_station_token` (id `n4sEAnrMqvG70nHp`) · **22 workflows migrados** do Baserow, incluindo `[UTIL] Refresh token Rd Station` · **set-tier** (`GADrDhIEi1Nv0Jn9`) reordenado com RD por último nos braços de formulário e WhatsApp · Sistema de token do CDP (`[UTIL] Obter credenciais da RD Station`) intacto · Workflows inativos que liam o Baserow (13) não foram alterados.